

SMART CAMPUS CANTEEN APP

Complete Semester Project Documentation

Step-by-Step Development Guide with Mockups, Schemas & Code

Tech Stack

Frontend: Expo + React Native
Backend: Node.js + Express
Database: MongoDB Atlas
Auth: JWT + bcryptjs

Project Scope

Duration: 6 Weeks
Screens: 14 Screens
API Routes: 18 Endpoints
Collections: 6 MongoDB

1. Project Overview

The Smart Campus Canteen App is a full-stack mobile application that digitizes the university food ordering experience. Students can browse the menu, place orders, and pay using a wallet system — while canteen admins manage inventory and track orders in real time.

Problem Statement: Students waste time queuing at the canteen without knowing what food is available or how long the wait will be. This app solves both problems with a digital ordering and queue management system.

1.1 Why This Project Is Strong for Viva

- Demonstrates MongoDB multi-document ACID transactions (advanced DB topic)
- Full CRUD operations across 6 collections
- JWT-based stateless auth — relevant to security topics
- Mobile + backend + database in one project
- Real-world problem with clean, verifiable features

1.2 Tech Stack

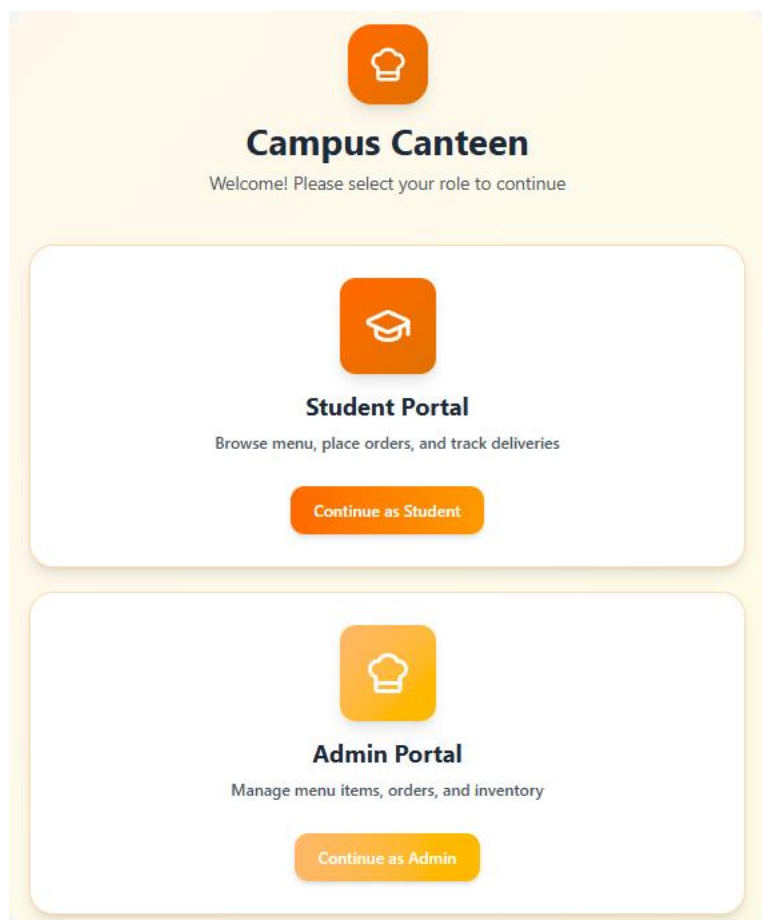
Layer	Technology	Purpose	
Frontend	Expo + React Native	Cross-platform mobile app (iOS + Android)	npm install
Navigation	Expo Router	File-based routing inside the app/ folder	npm install
HTTP Client	Axios	Makes API requests from Expo to Node.js backend	npm install
State Mgmt	React Context API	Auth state, cart state across screens	npm install
Backend	Node.js + Express	REST API server handling all business logic	npm install
Database	MongoDB Atlas	Cloud-hosted NoSQL database (free tier)	npm install
ODM	Mongoose	Schema modeling and query interface for MongoDB	npm install
Auth	JWT + bcryptjs	Stateless authentication with hashed passwords	npm install
Hosting	Render / Railway	Free backend hosting for deployed API	npm install
Version Control	GitHub	Source code + README + screenshots	npm install

2. App Screens & Wireframe Mockups

Below are wireframe mockups of all 14 screens. Each mockup shows the layout, key UI elements, and navigation flow. Build these screens one by one in Expo — do not try to style them perfectly at first.

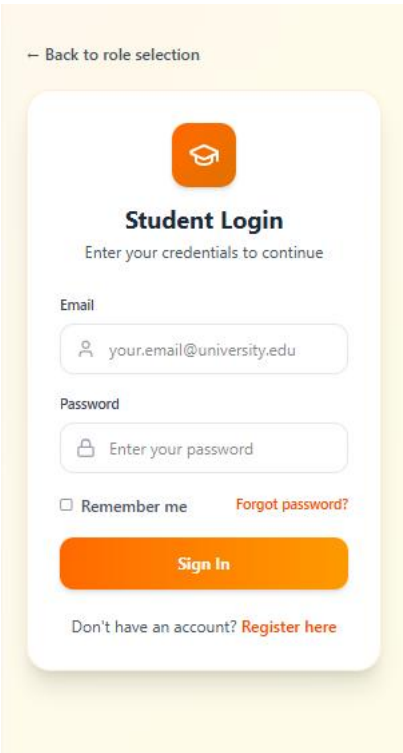
Design Tip: Use React Native Paper or NativeBase for ready-made components. Keep UI minimal in Week 2-3 and polish it in Week 6.

2.1 Welcome Screen

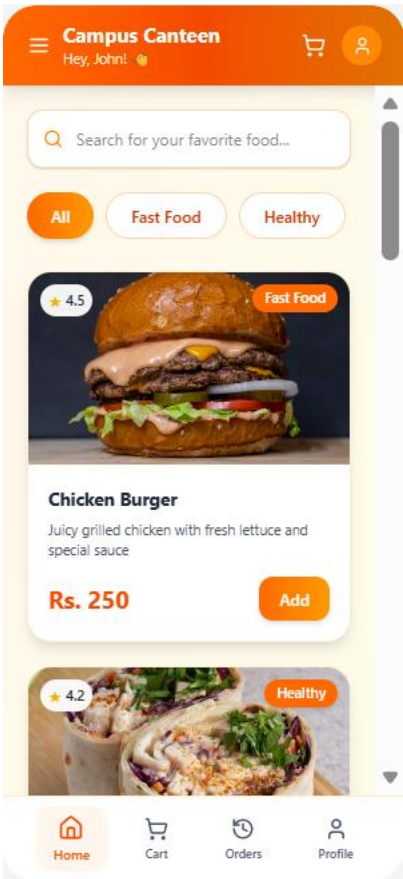


2.2 Student Screens

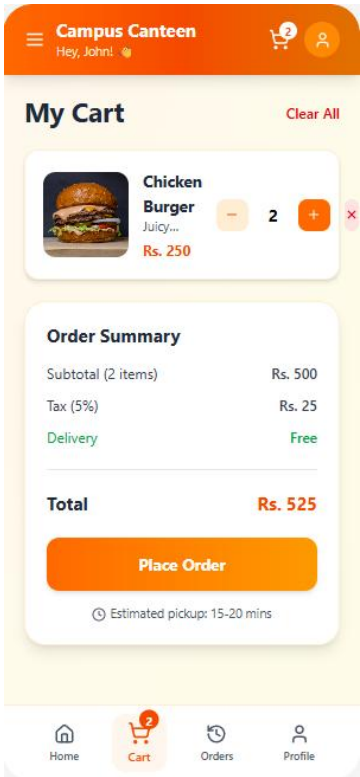
1. Login Screen



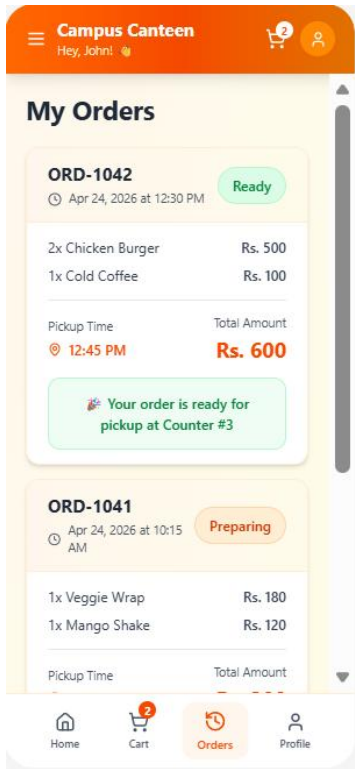
2. Home / Menu List Screen



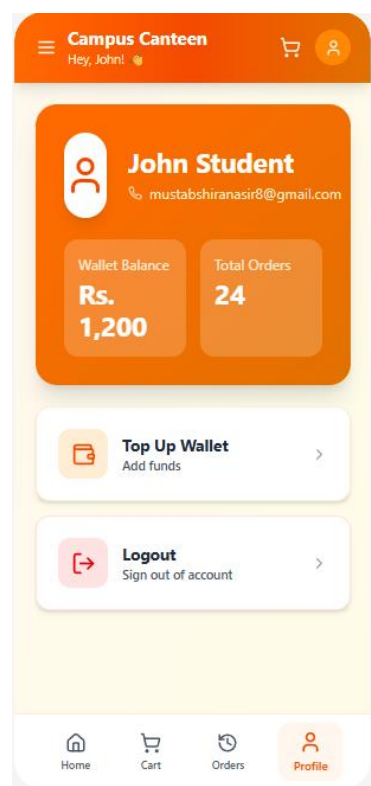
3. Cart Screen



4. My Orders Screen

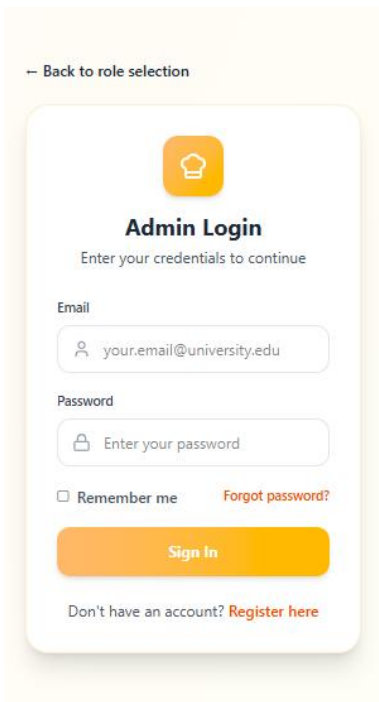


5. Profile Screen

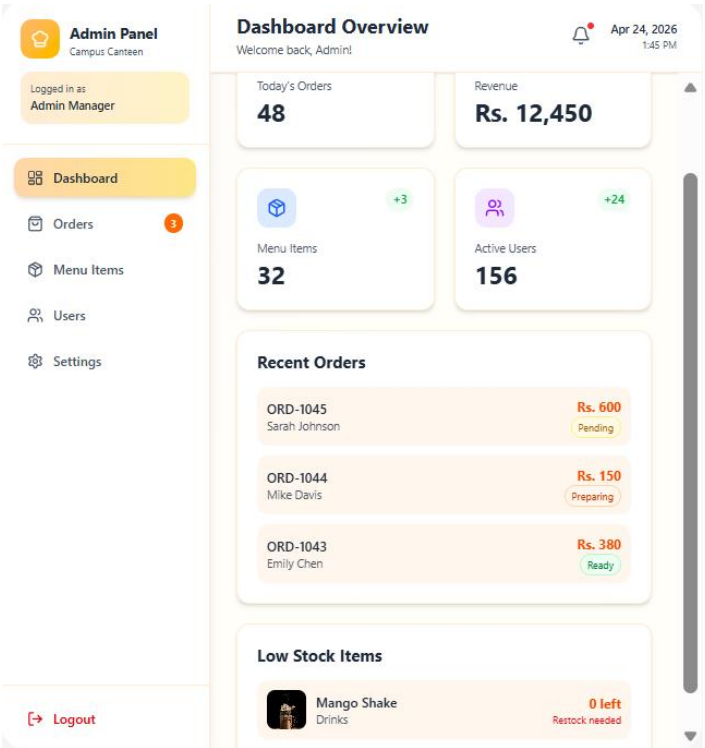


2.3 Admin Screens

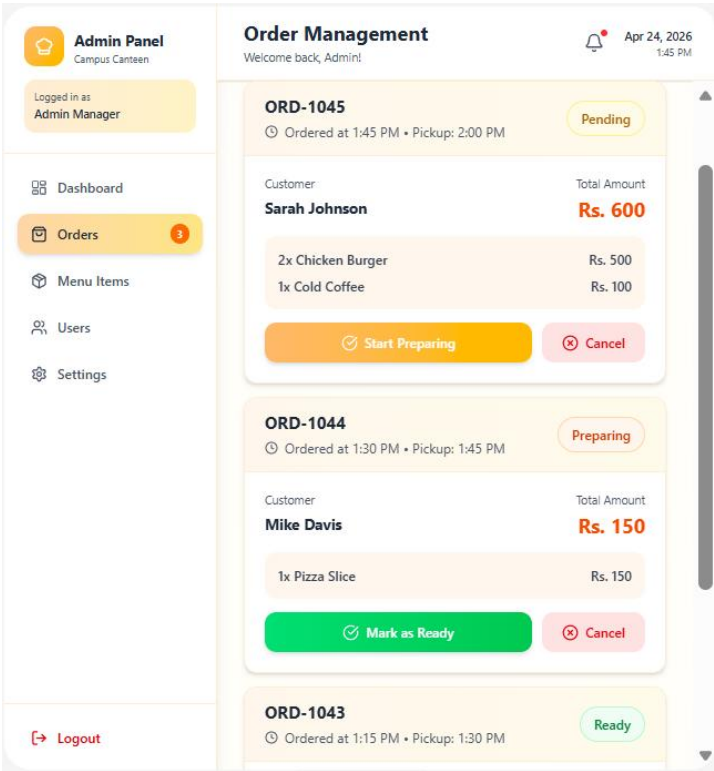
1. Admin Login Screen



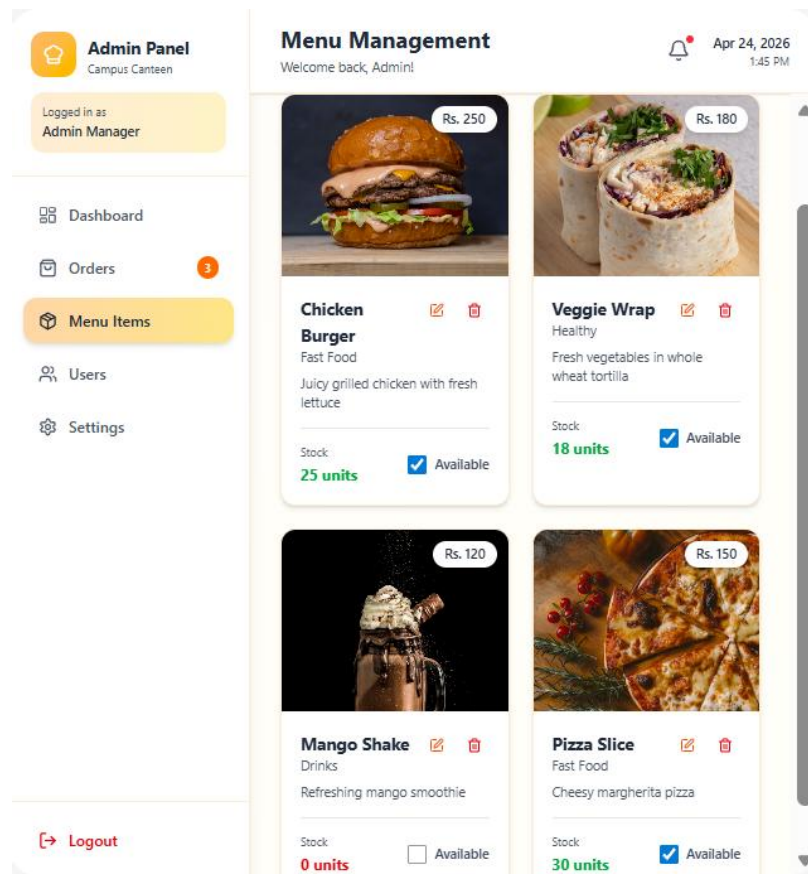
3. Admin Dashboard



5. Manage Orders Screen



4. Manage Menu Screen



2.2 Complete Screen List

Student Side (9 screens)

- 1. Splash / Welcome Screen
- 2. Login Screen
- 3. Register Screen
- 4. Home / Menu List Screen
- 5. Food Detail Screen
- 6. Cart Screen
- 7. Checkout Screen
- 8. My Orders Screen
- 9. Profile / Wallet Screen

Admin Side (5 screens)

- 1. Admin Login Screen
- 2. Admin Dashboard
- 3. Manage Menu Screen
- 4. Add / Edit Food Screen
- 5. Manage Orders Screen

3. Database Design — MongoDB Collections

Your database has 6 collections. Design them first before writing any backend code. The schemas below use Mongoose notation. Copy these directly into your model files.

Why MongoDB? MongoDB supports multi-document ACID transactions via sessions. This is critical for the checkout flow where an order, payment, stock update, and cart deletion must all succeed or all fail together.

3.1 users collection

Field	Type	Description
<code>_id</code>	<code>ObjectId</code>	Auto-generated unique ID (primary key)
<code>name</code>	<code>String</code>	Full name of student or admin
<code>email</code>	<code>String</code>	Unique email address — used for login
<code>passwordHash</code>	<code>String</code>	bcrypt hashed password — NEVER store plain text
<code>role</code>	<code>String</code>	'student' or 'admin' — controls access to routes
<code>walletBalance</code>	<code>Number</code>	Wallet balance in Rs. Default: 0
<code>createdAt</code>	<code>Date</code>	Auto set by timestamps: true in Mongoose

3.2 menuitems collection

Field	Type	Description
<code>_id</code>	<code>ObjectId</code>	Auto-generated unique ID
<code>name</code>	<code>String</code>	Display name e.g. 'Chicken Burger'
<code>price</code>	<code>Number</code>	Price in Rs e.g. 250
<code>category</code>	<code>String</code>	'Fast Food', 'Drinks', 'Snacks', 'Healthy'
<code>description</code>	<code>String</code>	Short description for the food detail screen
<code>stock</code>	<code>Number</code>	Current quantity available (decremented on order)
<code>isAvailable</code>	<code>Boolean</code>	Admin toggle to show/hide item from menu
<code>imageUrl</code>	<code>String</code>	URL to food image (use placeholder initially)

3.3 carts collection

Field	Type	Description
<code>_id</code>	ObjectId	Auto-generated unique ID
<code>userId</code>	ObjectId	Reference to users._id — one cart per user
<code>items</code>	Array	Array of { menuItemId, name, price, qty }
<code>totalAmount</code>	Number	Computed sum of all items x qty
<code>updatedAt</code>	Date	Timestamp of last cart change

3.4 orders collection

Field	Type	Description
<code>_id</code>	ObjectId	Auto-generated unique ID
<code>userId</code>	ObjectId	Reference to the student who placed order
<code>items</code>	Array	Snapshot of ordered items (copy from cart)
<code>totalAmount</code>	Number	Final order total in Rs
<code>paymentMethod</code>	String	'wallet' or 'cash'
<code>status</code>	String	'pending' → 'preparing' → 'ready' → 'delivered'
<code>pickupTime</code>	Date	Student-selected pickup time
<code>createdAt</code>	Date	Auto-set by Mongoose timestamps

3.5 payments collection

Field	Type	Description
<code>_id</code>	ObjectId	Auto-generated unique ID
<code>orderId</code>	ObjectId	Reference to orders._id
<code>userId</code>	ObjectId	Reference to users._id
<code>amount</code>	Number	Amount paid in Rs
<code>method</code>	String	'wallet' or 'cash'
<code>status</code>	String	'paid' or 'refunded'
<code>createdAt</code>	Date	Timestamp of payment

3.6 walletTransactions collection (optional)

Field	Type	Description
<code>_id</code>	ObjectId	Auto-generated unique ID

<code>userId</code>	<code>ObjectId</code>	Reference to users._id
<code>type</code>	<code>String</code>	'credit' (top-up) or 'debit' (order payment)
<code>amount</code>	<code>Number</code>	Transaction amount in Rs
<code>reason</code>	<code>String</code>	'Order Payment', 'Top Up', 'Refund'
<code>orderId</code>	<code>ObjectId</code>	Reference to order (null for top-ups)
<code>createdAt</code>	<code>Date</code>	Timestamp

4. Backend Setup — Step-by-Step

Step 1: Create Project Structure

Run the following commands in your terminal to create the full backend folder structure:

```
# Create root folder
mkdir smart-canteen-app
cd smart-canteen-app

# Create both sub-projects
mkdir backend mobile

# Initialize backend
cd backend
npm init -y

# Install all dependencies
npm install express mongoose cors dotenv bcryptjs jsonwebtoken
npm install --save-dev nodemon

# Create folder structure
mkdir -p src/config src/controllers src/middleware src/models src/routes
```

Step 2: Create server.js — Entry Point

This is the first file you write. It starts the Express server and connects MongoDB:

```
// backend/server.js
require('dotenv').config();
const app = require('./src/app');
const connectDB = require('./src/config/db');

const PORT = process.env.PORT || 5000;

connectDB().then(() => {
  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
  });
});
```

Step 3: Create db.js — MongoDB Connection

```
// backend/src/config/db.js
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log('MongoDB Connected');
  } catch (error) {
    console.error('DB connection failed:', error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Step 4: .env File — Environment Variables

```
# backend/.env
PORT=5000
MONGODB_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/canteen
JWT_SECRET=your_super_secret_key_here
```

Important: Never commit .env to GitHub. Add it to .gitignore. Create a .env.example with empty values for documentation.

Step 5: Create User Model

```
// backend/src/models/User.js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  passwordHash: { type: String, required: true },
  role: { type: String, enum: ['student', 'admin'], default: 'student' },
  walletBalance: { type: Number, default: 0 },
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

Step 6: Auth Controller — Register & Login

```
// backend/src/controllers/authController.js
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');

exports.register = async (req, res) => {
  try {
    const { name, email, password } = req.body;
    const exists = await User.findOne({ email });
    if (exists) return res.status(400).json({ message: 'Email already registered' });

    const passwordHash = await bcrypt.hash(password, 10);
    const user = await User.create({ name, email, passwordHash });

    res.status(201).json({ message: 'User registered successfully', userId:
user._id });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email });
    if (!user) return res.status(401).json({ message: 'Invalid credentials' });

    const match = await bcrypt.compare(password, user.passwordHash);
    if (!match) return res.status(401).json({ message: 'Invalid credentials' });

    const token = jwt.sign(
      { userId: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '7d' }
    );

    res.json({ token, user: { id: user._id, name: user.name, role: user.role } });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};
```

5. Complete API Endpoint Reference

Build these endpoints in this order. Test each one in Postman or Thunder Client before moving to the next.

5.1 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register new student account
POST	/api/auth/login	Public	Login and receive JWT token
GET	/api/auth/me	JWT	Get current logged-in user profile

5.2 Menu Endpoints

Method	Endpoint	Auth	Description
GET	/api/menu	Public	Get all available menu items
GET	/api/menu/:id	Public	Get single menu item by ID
POST	/api/menu	Admin	Add new menu item
PUT	/api/menu/:id	Admin	Update menu item details
DELETE	/api/menu/:id	Admin	Delete menu item
PATCH	/api/menu/:id/toggle	Admin	Toggle isAvailable on/off

5.3 Cart Endpoints

Method	Endpoint	Auth	Description
GET	/api/cart	JWT	Get current user's cart
POST	/api/cart/add	JWT	Add item to cart
PUT	/api/cart/update	JWT	Update item quantity in cart
DELETE	/api/cart/item/:id	JWT	Remove item from cart
DELETE	/api/cart/clear	JWT	Clear entire cart

5.4 Order Endpoints

Method	Endpoint	Auth	Description
POST	/api/orders/checkout	JWT	Place order (runs MongoDB transaction)
GET	/api/orders/my	JWT	Get all orders for current user
GET	/api/orders	Admin	Get all orders (admin view)
PATCH	/api/orders/:id/status	Admin	Update order status

5.5 Wallet & Payment Endpoints

Method	Endpoint	Auth	Description
GET	/api/wallet	JWT	Get current wallet balance + transactions
POST	/api/wallet/topup	JWT	Add funds to wallet (mock for now)
GET	/api/payments/my	JWT	Get payment history for user

6. MongoDB Transaction — Checkout Flow

This is the most important backend feature. It demonstrates advanced database usage and is what will impress your examiner the most.

Key Concept: A MongoDB transaction uses a session. All database operations inside the session are atomic — either ALL succeed and get committed, or ONE fails and EVERYTHING is rolled back. No partial data remains.

6.1 Transaction Step-by-Step Flow

1	Read Cart	Fetch userId cart from carts collection
2	Verify Stock	Check each menuItem stock >= qty ordered
3	Create Order	Insert new document into orders collection
4	Reduce Stock	Decrement stock field on each menuItem
5	Create Payment	Insert document into payments collection
6	Deduct Wallet	Subtract amount from user.walletBalance
7	Clear Cart	Delete cart document for this user
✓	COMMIT	All changes saved atomically

6.2 Checkout Controller Code

```
// backend/src/controllers/orderController.js
const mongoose = require('mongoose');
const Order = require('../models/Order');
const Cart = require('../models/Cart');
const MenuItem = require('../models/MenuItem');
const Payment = require('../models/Payment');
const User = require('../models/User');

exports.checkout = async (req, res) => {
  const { paymentMethod, pickupTime } = req.body;
```

```
const userId = req.user.userId;
const session = await mongoose.startSession();

try {
  await session.withTransaction(async () => {

    // Step 1: Get user's cart
    const cart = await Cart.findOne({ userId }).session(session);
    if (!cart || cart.items.length === 0)
      throw new Error('Cart is empty');

    // Step 2: Verify stock for all items
    for (const item of cart.items) {
      const menuItem = await MenuItem.findById(item.menuItemId).session(session);
      if (!menuItem || menuItem.stock < item.qty)
        throw new Error(`${item.name} is out of stock`);
    }

    // Step 3: Create order
    const order = await Order.create([
      {
        userId, items: cart.items,
        totalAmount: cart.totalAmount,
        paymentMethod, pickupTime, status: 'pending'
      }
    ], { session });

    // Step 4: Reduce stock
    for (const item of cart.items) {
      await MenuItem.updateOne(
        { _id: item.menuItemId },
        { $inc: { stock: -item.qty } },
        { session }
      );
    }

    // Step 5: Create payment record
    await Payment.create([
      {
        orderId: order[0]._id, userId,
        amount: cart.totalAmount,
        method: paymentMethod, status: 'paid'
      }
    ], { session });

    // Step 6: Deduct wallet (only if wallet payment)
    if (paymentMethod === 'wallet') {
      await User.updateOne(
        { _id: userId },
        { $inc: { walletBalance: -cart.totalAmount } },
        { session }
      );
    }

    // Step 7: Clear cart
    await Cart.deleteOne({ userId }, { session });
  });
}
```

```
    res.status(201).json({ message: 'Order placed successfully' });
  } catch (error) {
    res.status(400).json({ message: error.message });
  } finally {
    await session.endSession();
  }
};
```

Viva Tip: Be ready to explain: 'What happens if Step 4 fails?' Answer: 'The session aborts the entire transaction. The order created in Step 3 is rolled back automatically. No partial data exists in the database.'

7. Expo Frontend — Step-by-Step Setup

Step 1: Create Expo App

```
# From smart-canteen-app/ root
cd mobile
npx create-expo-app@latest .

# Install navigation and HTTP libraries
npx expo install expo-router react-native-safe-area-context
npx expo install react-native-screens
npm install axios @react-native-async-storage/async-storage
```

Step 2: Create API Service File

This file centralizes all API calls. Import it in your screens instead of writing fetch/axios code everywhere:

```
// mobile/services/api.js
import axios from 'axios';
import AsyncStorage from '@react-native-async-storage/async-storage';

const BASE_URL = 'http://YOUR_IP:5000/api'; // use your machine IP

const api = axios.create({ baseURL: BASE_URL });

// Auto-attach JWT token to every request
api.interceptors.request.use(async (config) => {
  const token = await AsyncStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;
```

Step 3: Auth Context

Wrap your app with AuthContext so any screen can access the current user:

```
// mobile/context/AuthContext.js
import { createContext, useContext, useState, useEffect } from 'react';
```

```
import AsyncStorage from '@react-native-async-storage/async-storage';

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(null);

  useEffect(() => {
    AsyncStorage.getItem('token').then(t => {
      if (t) setToken(t);
    });
  }, []);

  const login = async (userData, jwtToken) => {
    setUser(userData);
    setToken(jwtToken);
    await AsyncStorage.setItem('token', jwtToken);
  };

  const logout = async () => {
    setUser(null);
    setToken(null);
    await AsyncStorage.removeItem('token');
  };

  return (
    <AuthContext.Provider value={{ user, token, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => useContext(AuthContext);
```

8. Complete Project Folder Structure

This is the exact folder structure you should have by Week 5. Create folders as you build each feature.

```
smart-canteen-app/  
  backend/  
    src/  
      config/  
        db.js           ← MongoDB connection  
      controllers/  
        authController.js  
        menuController.js  
        cartController.js  
        orderController.js  
      middleware/  
        auth.js         ← JWT verify middleware  
      models/  
        User.js  
        MenuItem.js  
        Cart.js  
        Order.js  
        Payment.js  
      routes/  
        authRoutes.js  
        menuRoutes.js  
        cartRoutes.js  
        orderRoutes.js  
      app.js            ← Express app setup  
      server.js         ← Entry point  
    .env  
    .env.example  
    package.json  
  mobile/               ← Expo app  
    app/  
      (auth) /  
        login.jsx  
        register.jsx  
      (student) /  
        index.jsx       ← Home/Menu  
        cart.jsx  
        checkout.jsx  
        orders.jsx  
        profile.jsx  
      (admin) /  
        dashboard.jsx  
        menu.jsx  
        orders.jsx  
    components/  
    services/           ← API calls (axios)
```

```
context/          ← AuthContext
README.md
.gitignore
```

9. Week-by-Week Development Roadmap

Follow this roadmap strictly. Do not jump ahead to a later week's tasks. The sequence matters — each week builds on the previous one.

Timeline	Deliverables & Tasks
Week 1 Backend Setup	Node.js project init, MongoDB Atlas connection, .env config, test route GET /api/test working
Week 2 Auth & Models	User model (Mongoose), POST /register, POST /login, JWT generation, bcrypt password hashing
Week 3 Menu APIs + Expo Start	MenuItem model, full CRUD for menu, create Expo app with create-expo-app, Login + Menu screens
Week 4 Cart & Order APIs	Cart model, add/update/remove cart items, Order model, POST /checkout with MongoDB transaction
Week 5 Admin Panel + Frontend Connect	Admin screens: Dashboard, Manage Menu, Manage Orders. Connect all Expo screens to real APIs.
Week 6 Polish & Submit	Testing, fix bugs, write README, add screenshots to GitHub, record demo video, submit

9.1 Day-by-Day Breakdown — Week 1

Day	Task
Day 1	Create GitHub repo, mkdir smart-canteen-app, mkdir backend mobile, npm init -y
Day 2	Install dependencies, create server.js + app.js + db.js, connect MongoDB Atlas, test GET /api/test
Day 3	Create User model, write authController.js (register + login), create authRoutes.js
Day 4	Test register and login with Postman, create auth middleware (JWT verify), test protected route
Day 5	Create MenuItem model, write menu CRUD endpoints, test all 5 menu routes in

	Postman
Day 6	Create Cart model + controller, test add/update/remove cart, verify totalAmount calculation
Day 7	Review all code, commit to GitHub, write progress in README

10. GitHub Setup & Submission Checklist

10.1 .gitignore

```
# backend/.gitignore
node_modules/
.env

# mobile/.gitignore
node_modules/
.expo/
*.jks
*.keystore
```

10.2 README.md Sections

Your README is the first thing graders and employers see. Include all of these:

README Section	What to Write
Project Title & Badge	Smart Campus Canteen App + tech stack badges
Problem Statement	Why students need this (1-2 sentences)
Features	Bullet list of all working features
Screenshots	Inline images of 4-6 app screens
Tech Stack	Table of frontend/backend/DB technologies
Database Schema	ERD image or table of your 6 collections
API Endpoints	Table of all 18 routes with method and description
Transaction Explanation	1 paragraph explaining the checkout transaction
How to Run Locally	Step by step: clone, npm install, set .env, npm start
Team Members	Names and student IDs

10.3 Final Submission Checklist

- GitHub repo is public and has correct folder structure (backend/ + mobile/)
- All commits have meaningful messages (not 'fix' or 'update')

- README has screenshots, schema, and API table
- .env.example exists in backend/ with empty values
- MongoDB checkout transaction is working and tested
- Admin can add/edit/delete menu items
- Student can register, login, add to cart, and place order
- Order status updates are working from admin side
- App runs on physical device or emulator without crashes
- Demo video recorded and link added to README

11. Common Mistakes to Avoid

✗ DO NOT DO THIS

- Start with UI design before backend works
- Store plain text passwords (ALWAYS use bcrypt)
- Commit .env file to GitHub
- Try to build wallet/payments/analytics in Week 1
- Use Snack as your main project workflow
- Add real payment gateway (not needed for semester)
- Build admin dashboard before basic student flow works
- Start coding without designing MongoDB schema first

✓ DO THIS INSTEAD

- Backend works first, then connect Expo
- Always hash with bcrypt before saving to DB
- Add .env to .gitignore immediately
- Complete MVP first, add features only after
- Use local Expo project with create-expo-app
- Use mock wallet top-up — simple number update
- Student flow first: login → menu → cart → order
- Design all 6 collection schemas on paper first

12. Quick Reference — npm Commands

```
# Backend (from backend/)
```

```
npm run dev          # Start with  
nodemon              #  
npm start            # Start production  
node src/server.js   # Direct start
```

```
# Frontend (from mobile/)
```

```
npx expo start        # Start Expo dev  
npx expo start --tunnel # Use with  
backend               #  
npx expo android      # Run on Android
```

Final Advice: A working project with 5 features is far better than a half-broken project with 15. Focus on quality over quantity. Your transaction checkout alone will demonstrate more database knowledge than 10 basic CRUD screens.